2021 IEEE 21st International Conference on Software Quality, Reliability and Security Companion (QRS-C) | 978-1-6654-7836-6/21/$31.00 ©2021 IEEE | DOI: 10.1109/QRS-C55045.2021.00017

MQP: Mutants Quality Prediction for Cost-Effective Mutation Testing

Xingya Wang[1,2], Shiyu Zhang[1], Fangxiao Liu[3], Lichao Feng[1], Zhihong Zhao[1,*]
[1]*School of Computer Science and Technology, Nanjing Tech University, Nanjing, China*
[2]*Command and Control Engineering College, Army Engineering University of PLA, Nanjing, China*
[3]*State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, China*
*corresponding author: zhaozhih@njtech.edu.cn

***Abstract*—Mutation testing simulates typical software defects by executing mutation operations. It can effectively measure the defect detection capabilities of a test-suite. However, in the mutation testing, we must generate, compile, and execute plenty of mutants, which causes substantial computing costs. Mutants selection can reduce the number of mutants but cannot avoid compiling and executing all mutants. Therefore, mutation testing suffers an efficiency problem, making it hard to use in practice. In this paper, we propose a Mutants Quality Prediction method, MQP, which relies on the static features of the source program, test-suite, and mutants to predict the quality of a test-suite. MQP avoids compiling and executing all mutants during mutants selection and thus can save expensive computing costs. Then, we use MQP as the fitness function and select the evolutionary optimization strategy, Genetic Algorithm, to generate a high-quality mutant subset. We empirically studied MQP on nine widely used items. The experimental results show that MQP achieves a good predictive effect. Its accuracy rate can reach 98.30%, while the average absolute error is only 0.74%. Moreover, the reduction efficiency of the MQP-based mutation testing is about 20 times that of the random-based selective mutation testing.**

***Index Terms*—mutation testing, mutants quality prediction, mutant reduction**

I. Introduction

Defect detection ability is an essential aspect of test-suite quality measurement [1]. By simulating typical software defects, mutation testing can effectively measure defect detection capabilities [2][3]. In mutation testing, testers should generate plenty of mutants. However, even for the small-scale Siemens program (513 lines of non-comment code), there will be tens of thousands of mutants (23,847) [4]. For each mutant, testers should compile and execute it on all tests. Compiling and running these mutants will inevitably consume much testing time, resulting in efficiency problems and usability problems w.r.t. mutation testing [5][6][7]. Therefore, the efficiency of mutation testing needs to be optimized to improve its usability in practice. Selective mutation testing is one conventional way to reduce the time-cost of mutation testing. It aims to selectively generate a representative mutant subset with similar defects detection capability to all the mutants. Usually, testers rely on Quality Score (QS) to measure the representative degree of a mutant subset [1]. The higher QS of the subset, the more representative it is. However, selective mutation testing still suffers an efficiency problem. Due to the calculation of QS, it needs to collect executing information such as code coverage and thus cannot avoid the time-consuming process of a program running [8].

This paper proposes the Mutants Quality Prediction (MQP) method, which aims to obtain the QS of a mutant subset without running any mutants. Precisely, we extract the static features from the source program, the test-suite, and its high-QS mutants, including both stubborn and adequate features. After that, we build the QS prediction model by the machine learning algorithm. Furthermore, we employ the Genetic Algorithem(GA) and MQP to evaluate each individual (i.e., mutant subset). Finally, we select the mutants in the mutant subset with the highest predictive QS. Our empirical study on nine widely used projects shows that: (1) the predictive accuracy of MQP can reach 98.30%, while its average absolute error is only 0.74%. (2) the reduction efficiency of the MQP-based mutation testing is about 20 times that of the random-based selective mutation testing. The main contributions of our work are as follows:

(1) we propose the Mutants Quality Prediction (MQP) method. We summarized the mutant subset's stubborn and adequate features with the high-quality score by a machine learning-based mutant quality measurement. It avoids the time-consuming process of mutant running during mutation testing.

(2) we empirically studied the efficiency of the MQP and MQP-based mutants reduction method. Precisely, we employ the GA algorithm to generate the optimal mutants reduced result searchingly. Experimental results indicate that our approach can reduce the testing time while ensuring mutation testing quality.

The remaining structure of this paper is as follows. Chapter 2 introduces the background of mutation testing. Chapter 3 details the features of high-quality mutants. Chapters 4 and 5 present MQP and the MQP-based mutant reduction approach. In Chapter 6, we carried out an empirical study. Chapter 7 introduces related work. Finally, the last chapter summarizes our work.

II. BACKGROUND

A. Overview Mutation Testing

Mutation testing refers to the process of program mutation to quantify the strength of the test-suite to support testing. It relies on mutation operators, which spread the infected

2693-9371/21/$31.00 ©2021 IEEE
DOI 10.1109/QRS-C55045.2021.00017

program state, to produce semantic program mutants [2][9]. If the test-suite recognizes the infected mutant, we consider the mutant to be killed. If the output of one mutant is entirely consistent with the source program, we call it an equivalent mutant. The ratio of the non-equivalent mutants killed by the test-suite to all the non-equivalent mutants, $\frac{|MUT_{kill}|}{|MUT|-|MUT_{eq}|}$, is defined as mutation score [10]. *TS* represents the test-suite, $|MUT|$ represents the number of all mutants, $|MUT_{kill}|$ presents the number of mutants killed, $|MUT_{eq}|$ represents the number of equivalent mutants.

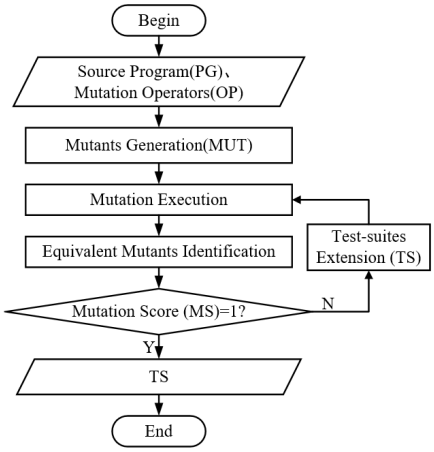

Fig. 1. General process of the mutation testing

Fig. 1 illustrates the general process of the mutation testing. Firstly, we prepare source program *PG*, test-suite *TS*, and mutation operators *OP*. Then, *PG* is trained with *OP* to generate the mutants *MUT*. We execute *TS* on the *MUT* and deletes all equivalent mutants. At last, the mutation score *MS* is calculated. If *MS* equals 1, it means *TS* has the strong defect detection ability. Otherwise, *TS* needs to be extended.

B. Selective Mutation Testing

Since a program may generate plenty of mutants, it is usually costly to execute the entire mutants [11]. To reduce the costs, researchers have proposed selective mutation testing by selecting a representative subset from all the mutants [12]. Quality Score *QS* measures the extent to which a mutant subset represents the entire mutants [1]. The formula is $\frac{|MUT_{kill}^{TS_{sub}}|}{|MUT|}$. $|MUT_{kill}^{TS_{sub}}|$ represents the number of mutants in the subset killed by the test-suite. $|MUT|$ denotes $MUT's$ size.

Fig. 2 illustrates the general process of the random-based selective mutation testing. Firstly, we initialize the mutants subset MUT_{sub} and randomly select one mutant *m* in the mutants *MUT* to MUT_{sub}. Then, we perform test-suite extension TS_{sub} until all mutants in MUT_{sub} are killed. After that, we test MUT by TS_{sub} and calculate the quality score of MUT_{sub}. If $QS(MUT_{sub})$ highers than the expected quality score $QS_{expected}(MUT_{sub})$,mutants selection stops. Otherwise, we repeat the former steps.

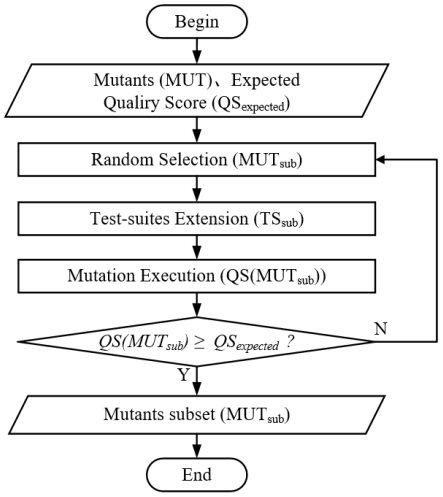

Fig. 2. General process of the random-based selective mutation testing

TABLE I
STUBBORNNESS FEATURES

type	**name and formula**
complexity	$cf = \frac{\sum_i^{\|MUT\|} ComplexFeature(mut_i)}{\|MUT\|}$
reachability	$nte = \frac{\sum_i^{\|MUT\|} numTestExecuted(mut_i)}{\|MUT\|}$
	$ntc = \frac{\sum_i^{\|MUT\|} numTestCovered(mut_i)}{\|MUT\|}$
	$nma = \frac{\sum_i^{\|MUT\|} numMutantAssertion(mut_i)}{\|MUT\|}$
	$nca = \frac{\sum_i^{\|MUT\|} numClassAssertion(mut_i)}{\|MUT\|}$

III. FEATURES OF HIGH-QUALITY MUTANTS

TABLE I and TABLE II summarize the stubbornness and adequacy features of the high-quality score mutants, respectively. Specifically, the complexity and the reachability are characterized as the stubbornness of mutants, while mutation operator and mutating position adequacy are characterized as the adequacy of mutants.

A. Stubbornness of Mutants

1) Complexity: Software indicators, including measuring software complexity [13], evaluating software maintainability [14], and measuring software quality [15], are used to measure quantifiable software system features. The more vast and complex the source program in mutation testing, the more difficult the infected program state spreads. In this paper, a

TABLE II
ADEQUACY FEATURES

type	**name and formula**
operator	$not = \frac{numOperaterType(MUT)}{\|OpType\|}$
position	$nc = \frac{numClass(MUT)}{\|class\|}$
	$nm = \frac{numMethod(MUT)}{\|Md\|}$
	$nmt = \frac{numMethodType(MUT)}{\|MdType\|}$

total of 89 software metrics are used in [16][**22**][**23**]. We use the average value of each indicator in the mutants to characterize the complexity features.

2) Reachability: In mutation testing, if the test-suites fail to reach the mutating statement, mutation ratio is high [17]. Therefore, reachability is used to reflect the stubbornness of mutation. The higher the degree of reachability is, the lower the stubbornness of the mutants is. We calculate the average of four indicators to characterize the reachability features.

B. Adequacy of Mutants

1) Adequacy of Mutation Operator: The effectiveness of mutation directly depends on the mutation operators used [18]. The more mutation operators involved in mutants, the more fault injection of the source program, which means the more adequate the mutants are. Therefore, the greater adequacy of the mutation operator, the higher adequacy of the mutants.

2) Adequacy of Mutation Position: The adequacy of mutants is also related to the mutation position of each mutant [18]. The more mutation statements are dispersed in the source program, the more widespread distribution of fault injection.

IV. Mutants Quality Prediction

To reduce the high computing costs of mutation testing, we propose the mutants quality prediction(MQP) method based on the analysis of program static features. Firstly, we generate the mutants. Then, we statically extract the feature of high-quality mutants. After that, we build the mutants quality prediction model by these feature. Finally, we obtain the predictive model that can evaluate the quality of the mutants.

The input of MQP is the source program *PG* and the test-suite *TS*. The final outputs are the mutants *MUT* and mutants quality prediction model (*MQPModel*). The critical process is as follows. (1) Mutants Generation. The *PG* is mutated to generate *MUT*. Then, *PG*, *MUT*, and *TS* are input into the features extraction program. (2) Predictive features Extraction. We randomly select *n* mutants subsets MUT_{sub} from *MUT* and $QS(MUT_{sub})$ from *PG*, *TS* and *n* MUT_{sub}, generating training set *TrainingSet*. (3) Predictive Model Construction. We input *TrainingSet* into the machine learning algorithm and adjust parameters to generate *MQPModel*. Based on the input mutants subset (MUT_{sub}), *MQPModel* can precisely predict the QS of this subset ($QS(MUT_{sub})$). *MUT* and *MQPModel* are final output for mutants reduction.

V. MQP-based Mutants Reduction

Static program analysis by predicting the SQ of mutants can effectively reduce the computational costs. However, the process of selecting an optimal mutant subset from a large number of mutants is still very time-consuming. MQP is applied to the genetic algorithm as a fitness function to save the computing costs of the entire reduction process. The input of the method is the *MUT* and the *MQPModel*. The final output is the reduced mutants subset MUT_{sub}. It is mainly divided into three steps.

TABLE III
EXPERIMENT SUBJECTS

name	version	#stmt	#test	coverage	#ne-mutants
joda	v2.10.8	14,956	4,238	90.37%	7,753
lafj	v0.4.0	2,943	245	63%	1,527
lang	v2.1.1	12,372	2,362	97.16%	8,748
msg	v0.6.4	4,720	973	69%	1,666
exp4j	v0.4.8	642	311	95%	341
text	v1.0	3,097	476	96%	1,880
io	v2.5	5,182	1,032	88.50%	2,433
linq4j	v0.4	6,133	221	48%	1,265
collections	v4.0	11,555	13,677	85%	2,392

1) Population Initialization: The population individual represents the mutants subset MUT_{sub}. It is encoded in the form of a binary bit array. In the binary of *n* bit, *0* means that the corresponding mutant is not selected. In contrast, *1* means that the corresponding mutant is selected. In the *MUT*, *m* mutants are randomly selected to form the initial population $p^0 = \{MUT_1^0, MUT_2^0, ..., MUT_M^0\}$.

2) Quality Assessment: *MQPModel* is used to evaluate the fitness quality of the current population $p^i = \{MUT_1^i, MUT_2^i, ..., MUT_M^i\}$. After obtaining the quality evaluation results $QS^i = \{QS_1^i, QS_2^i, ..., QS_m^i\}$, we save the individuals $MUT_j^i = \{j|QS_j^i = \{QS_1^i, QS_2^i, ..., QS_m^i\}_{max}\}$ with the highest fitness in the current population. If all the individuals are met the requirements, the one with the highest fitness in *Res* is output. That is, the high-quality mutants subset (MUT_{sub}). Otherwise, we proceed to the next step.

3) Population Evolution: Among all the individuals, the first *m* individuals with high fitness values are retained to form a new generation of population. The new generation population is input to the previous step for iteration until the termination condition is met. For the crossover mentioned above, the specific action is recombining and allocating the genes on the parent and mother individuals. For the above mutation operation, the specific steps are to modify the genes in the population [19].

VI. VERIFICATION EXPERIMENT

A. Research Questions

Q1: How about the accuracy and efficiency of MQP?

Q2: How do the features of mutation stubbornness and mutation adequacy contribute to the mutants quality prediction model?

Q3: How about the accuracy and efficiency of MQP-based mutants reduction?

B. Experiment Subjects

This paper takes nine widely used items [8][20] as the experimental subjects of empirical research. TABLE III shows the basic information of 9 subjects. We use all 18 mutation operators of Pitest to generate the entire set of mutants. Then, we run all test-suites on them and obtain a set of mutants that these test-suites (MUT_{killed}) can kill. Referring to the previous work [4][21][22], we effectively identify the equivalent mutants. In the experiment, the mutants that any test-suite

cannot kill are reduced as equivalent mutants. Otherwise, the mutants are considered non-equivalents (MUT_{ne}).

C. Evaluation Indicators

This section introduces the corresponding evaluation indicators for the three questions raised in VI-A. The specific evaluation indicators are as follows.

Root Mean Square Error (RMSE) is used to measure the deviation between the predicted value and the actual value. Mean Absolute Error (MAE) represents the average value of the absolute error between the predicted and the observed value. Goodness of Fit (R2) represents the difference between the predicted and the observed value to judge the fitting effect of the model. Reduction Ratio of Mutants (MRR) refers to the ratio of the number of reduced mutants ($|MUT_{ne}| - |MUT_{sub}|$) to the number of all non-equivalent mutants ($|MUT_{ne}|$). Test-suite Reduction Ratio (TRR) refers to the ratio of the reduced number of test-suites in the reduced mutants subset ($|TS_{complete}| - |TS^{MUT_{sub}}_{complete}|$) to the test-suite pool ($|TS_{complete}|$). Quality Score (QS) refers to the $MS(MUT_{ne}, TS_{sub})$ of the TS_{sub}. Running Time (RT) refers to the CPU running time of the reduction, in seconds.

D. Experimental Design

In this section, experiments are designed for the above three questions. The specific design is as follows.

For Q1, it is mainly divided into three steps.

(1) Construction of *TrainingSet*. We construct the *TrainingSet* concerning the reduction ratio in mutation reduction [2]. First, the reduction ratio array *ReduceRatios*=[*10%, 20%,..., 90%*] is set. It is executed *n* times with different reduction ratios to ensure the uniform distribution of *TrainingSet*. Then, we traverse each mutant subset MUT^i_{sub} to the feature vector by extracting the relevant features. We select the test-suite of MUT^i_{sub} from $TS_{complete}$, whose MS on the MUT^i_{sub} should be close to 1. After that, we select the MS of the test-suite in all the mutants to form the label value. Finally, $n * |ReduceRatios|$ samples are obtained for the *TrainingSet*.

(2) Selection of predictive model construction methods. We choose the ridge regression algorithm [23], the support vector machine regression algorithm [24], the nearest neighbor regression algorithm [25] and the random forest algorithm [26] to construct the quality prediction model. The performances of these prediction models are evaluated. We pick the optimal prediction model.

(3) Model training and evaluation. Firstly, the *TrainingSet* is input to the given prediction algorithm. We train the prediction model *MQPModel* with the grid search method [27] and automatically select the parameter with the highest score. Then, the 5-fold cross-validation method [8] is used to evaluate the effectiveness of all prediction models. Among them, we select the best prediction model.

For Q2, there are two steps.

(1) Two training sample datasets are constructed. Referring to Q1, two training sets are obtained. $TrainingSet_1$ only contains the stubborn features, while $TrainingSet_2$ contains the adequate features.

(2) Feature contribution evaluation. First, we select all mutation stubbornness features to construct a vector. The corresponding goodness of fit $R2_{stubborn}$ is recorded. Similarly, we select all mutation adequacy features to construct a vector. The goodness of fit $R2_{adequacy}$ is also recorded. Comparing $R2_{stubborn}$ and $R2_{adequacy}$, we can evaluate the contribution of mutation stubborn and adequate degree to *MQPModel*.

For Q3, there are three steps.

(1) Execute MQP-based mutants reduction. Firstly, we implement the MQP-based mutant reduction. We use the grid search method [27] to find the optimal number of iterations automatically. Second, we collect the number of mutants before and after reduction and calculate the mutation reduction ratio (MRR_{MQP}). Third, we select the test-suite ($TS^{MUT_{sub}}_{complete}$) of the reduced mutants subset. The QS (QS_{MQP}) and the test-suite reduction ratio (TRR_{MQP}) on all the mutants are calculated. We record the time ($Time_{MQP}$) consumed by the MQP-based mutants reduction.

(2) Execute RMR. Studies have shown that the random mutants selection method is better than the mutation operator selection method [4]. First, according to MRR_{MQP}, we reduce the set of mutants based on random selection [1]. The mutants reduction ratio (MRR_{RMR}) is recorded. Next, we select the test-suite ($TS^{MUT_{sub}}_{complete}$) of the reduced mutants subset. The QS (QS_{RMR}) of $TS^{MUT_{sub}}_{complete}$ and the test-suite reduction ratio (TRR_{RMR}) are calculated. Finally, the time ($Time_{RMR}$) consumed by the reduction method is recorded.

(3) Comparison of evaluation indicators. Firstly, to reduce the occurrence of unexpected results, we perform steps (1) and (2) *m* times, respectively. We collect MRR, TRR, QSM, and RT of the reduced mutants subset. Besides, we refer to [4] and set the value of *m* to 50, which is adequate to avoid unexpected results. Then, the collected average indicators are compared to evaluate the accuracy and efficiency of MQP-based mutants reduction.

E. Experimental Result

We summarize the R2 of the prediction models constructed on different experimental subjects using the four regression algorithms mentioned above. Comparing experimental results, we found the prediction model constructed by the random forest algorithm is better than others.

TABLE IV summarizes the experimental results of the optimal prediction model constructed by different subjects. From the table, it can be concluded that the R2 is above 0.95. The average R2 as high as 0.983. In addition, both RMSE and MAE are relatively low. Their average values are 0.0114 and 0.0074, respectively. The above observations show that MQP has good stability and high accuracy.

Fig. 3 shows the contribution of each category to the prediction model. The x-axis is the experimental subject. The y-axis is R2. It can be seen that the fit values are almost the same by the only features of stubbornness or adequacy. The

TABLE IV
Q1 RANDOM FOREST FORECAST MODEL EXPERIMENT RESULTS

Name	R2	RMSE	MAE
joda	0.9987	0.0050	0.0033
lafj	0.9642	0.0116	0.0072
lang	0.9983	0.0049	0.0032
msg	0.9722	0.0119	0.0067
exp4j	0.9591	0.0277	0.0187
text	0.9878	0.0106	0.0067
io	0.9946	0.0091	0.0064
linq4j	0.9744	0.0135	0.0081
collections	0.9975	0.0086	0.0064

above findings show that the features of each category play a key role in constructing mutants quality prediction model.

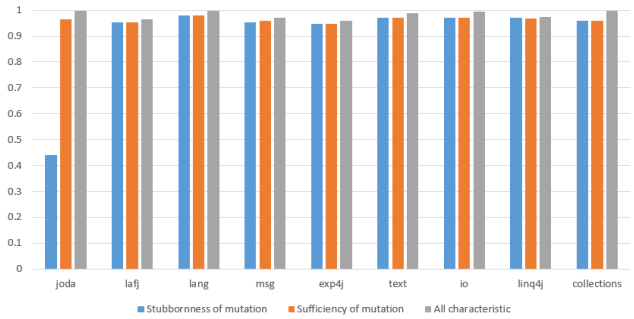

Fig. 3. Q2 Contributions of two types of features.

TABLE V summarizes the results of MQP-based mutants reduction and RMR under different experimental subjects. The most obvious is that these two methods have similar TRR and QS but a significant gap in RT. To compare the two difference, we analyze the RT of MQP-based mutants reduction and RMR separately. Based on the experimental results of 9 subjects, the average time consumed by RMR is about 21.13 times that of MQP-based mutants reduction. The conclusion is that MQP-based mutants reduction significantly reduces the running time and the computation costs than the RMR method.

VII. RELATED WORKS

The mutants reduction strategy aims to select representative subsets, reducing the size of the mutants and computing costs. Researchers such as Acree [28] pioneered the random mutation testing selection technique. Then, Mathur and Wong [29] conducted an empirical study on the mutants produced by the 22 mutation operators in Mothra [30] by randomly selecting mutants. Papadakis and Malevris [31] proved that random selection of 10%, 30%, 50% mutants could lead to approximately 26%, 13%, 7% failure loss. Zhang [4] constructed a set of test-suites that can kill randomly selected mutants subsets. Its lethality in all mutants is more than 99%. Researchers such as Gopinath [32] conducted empirical studies on large open-source programs. The results showed that selecting a few mutants can obtain similar MS as all mutants.

Machine learning has provided solutions in many areas to solve costs-related problems [33]. Zhang [8] first proposed the

TABLE V
Q3 RMR AND MQP-BASED MUTANTS REDUCTION EXPERIMENTAL RESULTS ANALYSIS

Name	MRR	Method	TRR	QS	RT
joda	0.3982	RMR	19.86%	0.9292	2,409
		MQP	20.60%	0.9313	104
lafj	0.4076	RMR	4.26%	0.9802	483
		MQP	9.93%	0.9902	60
lang	0.4036	RMR	10.59%	0.9646	2,532
		MQP	16.90%	0.9675	123
msg	0.4186	RMR	8.02%	0.9722	3,219
		MQP	9.09%	0.9814	61
exp4j	0.3987	RMR	16.48%	0.891	220
		MQP	20.88%	0.9359	35
text	0.3944	RMR	7.55%	0.9695	758
		MQP	10.94%	0.9761	51
io	0.4030	RMR	12.64%	0.9462	1,757
		MQP	18.03%	0.9499	58
linq4j	0.3979	RMR	7.70%	0.972	596
		MQP	8.88%	0.985	41
collections	0.4019	RMR	19.56%	0.8982	961
		MQP	20.44%	0.9193	49

predictive mutation testing method (PMT), which established a classification model that predicted the execution results of mutants without performing mutation testing. Researchers such as Mao [34] conducted extensive research based on PMT to evaluate the performance of predictive mutation testing. Experimental results show that the average prediction accuracy of practical items exceeds 0.85, which proves the effectiveness of PMT. Alireza [35] investigated the impact of uncovered mutants on the results of PMT research. Specifically, they found that such mutants significantly reduced the performance in terms of AUC. Then, a method based on ensemble learning is proposed to solve this problem.

VIII. CONCLUSION

Currently, mutants selection relies on multiple dynamic implements to calculate the QS of each selected mutants subset, which consumes lots of testing costs. In this paper, we introduce the ideas of static analysis and genetic algorithms into mutants reduction by constructing a quality prediction model to gain the QS. It searches for the best quality mutants subset by applying the genetic algorithm. This method has been verified on nine projects widely used in software testing. Experiments show that the prediction model constructed by the MQP method evaluates the goodness of fit as high as 98.30%, while the reduction time is only 1/20 of that of the RMR method. Both MQP and MQP-based mutants reduction perform well in terms of reducing the mutants and computing costs. It improves the efficiency of the mutation testing.

ACKNOWLEDGMENT

The work is partly supported by the General Project of Basic Natural Science in Colleges and Universities of Jiangsu Province (21KJB520027), the Key Project of University Education Information Research (2021JSETKT023), the Postgraduate Research Practice Innovation Program of Jiangsu Province (KYCX21_1140) and the Project of University-Industry Collaborative Education (202002180001).

REFERENCES

[1] Jie Zhang et al. "An empirical study on the scalability of selective mutation testing". In: *ISSRE*. 2014, pp. 277–287.

[2] Mike Papadakis et al. "Mutation testing advances: An analysis and survey". In: *Advances in Computers*. Vol. 112. 2019, pp. 275–378.

[3] Fevzi Belli et al. "Model-based mutation testing—approach and case studies". In: *Science of Computer Programming* 120 (2016), pp. 25–48.

[4] Lu Zhang et al. "Is operator-based mutant selection superior to random mutant selection?" In: *ICSE*. 2010, pp. 435–444.

[5] Vidroha Debroy and W Eric Wong. "Combining mutation and fault localization for automated program debugging". In: *JSS* 90 (2014), pp. 45–60.

[6] Yue Jia and Mark Harman. "An analysis and survey of the development of mutation testing". In: *TSE* 37.5 (2010), pp. 649–678.

[7] Dunwei Gong et al. "Mutant reduction based on dominance relation for weak mutation testing". In: *IST* 81 (2017), pp. 82–96.

[8] Jie Zhang et al. "Predictive mutation testing". In: *TSE* 45.9 (2018), pp. 898–918.

[9] Fevzi Belli, Christof J Budnik, and W Eric Wong. "Basic operations for generating behavioral mutants". In: *Mutation 2006-ISSRE Workshops*. IEEE. 2006, pp. 9–9.

[10] Lei Ma et al. "Deepmutation: Mutation testing of deep learning systems". In: *ISSRE*. 2018, pp. 100–111.

[11] René Just et al. "Are mutants a valid substitute for real faults in software testing?" In: *FSE*. 2014, pp. 654–665.

[12] Aditya P Mathur. "Performance, effectiveness, and reliability issues in software testing". In: *COMPSAC*. 1991, pp. 604–605.

[13] Tu Honglei, Sun Wei, and Zhang Yanan. "The research on software metrics and software complexity metrics". In: *IFCSTA*. Vol. 1. 2009, pp. 131–136.

[14] Don Coleman et al. "Using metrics to evaluate software system maintainability". In: *Computer* 27.8 (1994), pp. 44–49.

[15] Stephen H Kan. *Metrics and models in software quality engineering*. Addison-Wesley Professional, 2003.

[16] Alessandro Murgia et al. "Parameter-based refactoring and the relationship with fan-in/fan-out coupling". In: *ICSTW*. 2011, pp. 430–436.

[17] Willem Visser. "What makes killing a mutant hard". In: *ASE*. 2016, pp. 39–44.

[18] Marcio Eduardo Delamaro, Jeff Offutt, and Paul Ammann. "Designing deletion mutation operators". In: *ICST*. 2014, pp. 11–20.

[19] Man Zhao et al. "Optimization Scheduling Design of Monitoring Resources using a Process-improved Adaptive Genetic Algorithm". In: *ISSSR*. IEEE. 2021, pp. 167–177.

[20] Marinos Kintis et al. "How effective are mutation testing tools? An empirical analysis of Java mutation testing tools with manual analysis and real faults". In: *ESE* 23.4 (2018), pp. 2426–2463.

[21] Akbar Siami Namin, James Andrews, and Duncan Murdoch. "Sufficient mutation operators for measuring test effectiveness". In: *ICSE*. 2008, pp. 351–360.

[22] Lingming Zhang et al. "Operator-based and random mutant selection: Better together". In: *ASE*. 2013, pp. 92–102.

[23] Victor Rodriguez-Galiano et al. "Predictive modeling of groundwater nitrate pollution using Random Forest and multisource variables related to intrinsic and specific vulnerability: A case study in an agricultural setting (Southern Spain)". In: *STOTEN* 476 (2014), pp. 189–206.

[24] Gary C McDonald. "Ridge regression". In: *WIREs* 1.1 (2009), pp. 93–100.

[25] Ahmed Shokry et al. "Modeling and simulation of complex nonlinear dynamic processes using data based models: application to photo-Fenton process". In: *Computer Aided Chemical Engineering*. Vol. 37. Elsevier, 2015, pp. 191–196.

[26] Rinkaj Goyal, Pravin Chandra, and Yogesh Singh. "Suitability of KNN regression in the development of interaction based software fault prediction models". In: *Ieri Procedia* 6 (2014), pp. 15–21.

[27] Eugene Ndiaye et al. "Safe grid search with optimal complexity". In: *ICML*. 2019, pp. 4771–4780.

[28] Allen T Acree et al. *Mutation Analysis.* Tech. rep. Georgia Inst of Tech Atlanta School of Information and Computer Science, 1979.

[29] W Eric Wong and Aditya P Mathur. "Reducing the cost of mutation testing: An empirical study". In: *JSS* 31.3 (1995), pp. 185–196.

[30] James A Jones and Mary Jean Harrold. "Empirical evaluation of the tarantula automatic fault-localization technique". In: *ASE*. 2005, pp. 273–282.

[31] Mike Papadakis and Nicos Malevris. "An empirical evaluation of the first and second order mutation testing strategies". In: *ICSTW*. 2010, pp. 90–99.

[32] Rahul Gopinath et al. "How hard does mutation analysis have to be, anyway?" In: *ISSRE*. 2015, pp. 216–227.

[33] Muhammad Rashid Naeem et al. "Scalable mutation testing using predictive analysis of deep learning model". In: *IEEE Access* 7 (2019), pp. 158264–158283.

[34] Dongyu Mao, Lingchao Chen, and Lingming Zhang. "An extensive study on cross-project predictive mutation testing". In: *ICST*. 2019, pp. 160–171.

[35] Alireza Aghamohammadi and Seyed-Hassan Mirian-Hosseinabadi. "The Threat to the Validity of Predictive Mutation Testing: The Impact of Uncovered Mutants". In: *ArXiv* (2020).